



UNIVERSIDAD NACIONAL AUTÓNOMA DE  
MÉXICO

FACULTAD DE INGENIERÍA  
DIVISIÓN DE INGENIERÍA ELÉCTRICA  
INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO  
COMPUTADORA

## **REPORTE DE PRÁCTICA N° 08**

**NOMBRE COMPLETO:** Velazquez Caudillo Osbaldo

**N° de Cuenta:** 320341704

**GRUPO DE LABORATORIO:** 02

**GRUPO DE TEORÍA:** 06

**SEMESTRE 2026-1**

**FECHA DE ENTREGA LÍMITE:** 26/10/25

**CALIFICACIÓN:** \_\_\_\_\_

## REPORTE DE PRÁCTICA:

### 1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Antes de comenzar con la explicación de los ejercicios uno por uno, para que funcionara toda la lógica del control de las luces, lo primero fue crear vectores auxiliares tanto para los Spotlight como para los Pointlight así como también una variable contador para cada uno

```
//vector temporal de los pointsLight
PointLight pointLightstemp[MAX_POINT_LIGHTS];
//contador de los pointlight
int p = 0;
//vector temporal
SpotLight Spottemp[MAX_SPOT_LIGHTS];
//contador de los spotlight
int s = 0;
```

Estos arreglos nos ayudaran a saber exactamente que luces se le van a enviar al shader

```
shaderList[0].SetSpotLights(Spottemp,s);

shaderList[0].SetPointLights(pointLightstemp, p);
```

Como se puede observar a cada shaderList le enviamos su respectivo vector temporal a si como su contador para que sepa cuantas luces son las que tenemos, y este va a ocurrir únicamente después de que cumpla algunas de las condiciones propuestas, de esta forma es como se van a estar utilizando en los ejercicios.

1. Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del cofre de su coche y al abrir y cerrar el cofre ilumine en esa dirección.

Primero creamos el spotlight con sus valores iniciales.

```
GLfloat getmuevecofre() { return muevecofre; }  
  
GLfloat muevecarro;  
GLfloat muevecofre;
```

Creamos una función que es la que se va a encarga del movimiento del cofre.

```
//para mover cofre del carro  
if (key == GLFW_KEY_H)  
{  
    static int dir = -1;  
    theWindow->muevecofre += 2.0f * dir;  
  
    // tope al abrir (negativo)  
    if (dir < 0 && theWindow->muevecofre <= -55.0f) {  
        theWindow->muevecofre = -55.0f;  
        dir = +1;  
    }  
  
    // tope al cerrar (cero)  
    else if (dir > 0 && theWindow->muevecofre >= 0.0f) {  
        theWindow->muevecofre = 0.0f;  
        dir = -1; // al llegar, invierte: ahora a abrir  
    }  
}
```

Le asignamos la letra H la cual se va a encargar de todo el movimiento del cofre, después agregamos esta función a nuestro código que contiene main.

Creamos la luz para el cofre

```
//luz del cofre  
spotLights[1]= SpotLight(1.0, 0.5, 0.0,  
    5.0,1.0f,  
    0.0f,0.0f,0.0f,  
    1.0f,0.1f,0.0f,  
    2.0f,0.1f,0.02f,  
    30.0f);  
//spotlightCount++
```

```
//luz del cofre
if (mainWindow.getmuevecofre() <= 0.0f) {
    Spottemp[s++] = spotLights[1];
    //printf("El valor de s es: %d\n", s);
}
```

Con este condicional lo que haces es decir que, si el valor retornado de la función es menor o igual a 0 entonces que nos guarde en nuestro arreglo auxiliar esa luz en este caso específicamente la 0, para que se mande al Shader.

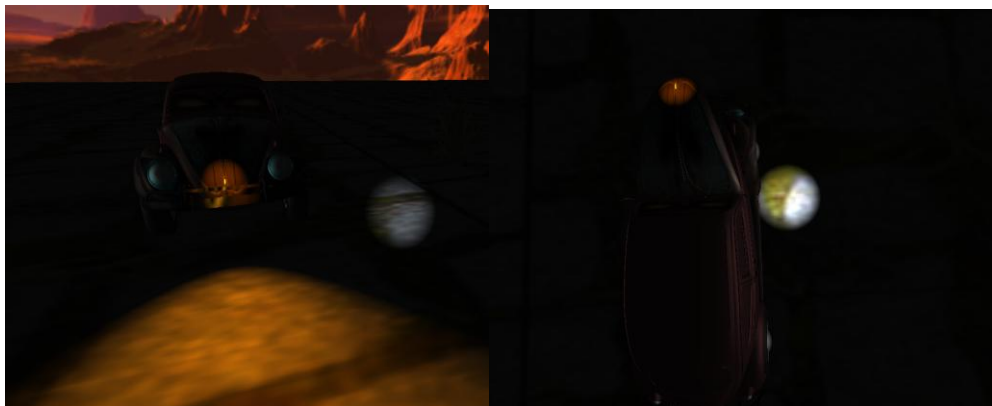
Asignamos la luz al cofre.

```
//Cofre
model = modelaux;
model = glm::translate(model, glm::vec3(2.4f, 0.3f, 0.0f));
model = glm::rotate(model, glm::radians(-8.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -glm::radians(mainWindow.getmuevecofre()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glm::vec3 cofrePos = glm::vec3(model * glm::vec4(0.0f, -1.02f, -5.0, 1));
glm::vec3 cofreDir = glm::normalize(-glm::vec3(model[2]));
spotLights[1].SetFlash(cofrePos, cofreDir);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_C.RenderModel();
```

Ahora creamos dos variables que nos ayudaran con la posición y con la dirección

La variable cofrePos convierte un punto local del cofre a posición global cofreDir toma el eje y lo normaliza para usarlo como dirección del spotlight. Así la luz queda totalmente jerárquica: sigue la pose del cofre.

## Ejecución





2. Agregar luz de tipo spotlight para el coche de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight hacia adelante y al retroceder ((mover con teclado hacia dirección de X positiva) ilumine con un spotlight hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán.

En este ejercicio implemente 3 funciones una de ellas para agregarle movimiento al carro y las otras dos como banderas.

```
GLfloat getmuevecarro() { return muevecarro; }
bool avanzar() { return avanza; }
bool retroceder() { return retrocede; }
```

La función `getmuevecarro()` se va a encargar de que se traslade el carro haciendo incremento y decrementos, dependiendo de si avanza o retrocede, mientras que las funciones `avanzar()` y `retroceder()` nos servirán como bandera para saber en momento realiza esa acción sin importar en donde este ubicado el coche, por lo cual lo único que nos retorna es un valor booleano true o false dependiendo el caso.

```
//para mover carro
if (key == GLFW_KEY_F)
{
    theWindow->muevecarro += 1.0;
    theWindow->avanza = true;
}
if (key == GLFW_KEY_F && action == GLFW_RELEASE) {
    theWindow->avanza = false;
}
if (key == GLFW_KEY_G)
{
    theWindow->muevecarro -= 1.0;
    theWindow->retrocede = true;
}
if (key == GLFW_KEY_G && action == GLFW_RELEASE) {
    theWindow->retrocede = false;
}
```

Para el movimiento utilizamos la letra F entonces si se presiona esa letra la variable se incrementará en 1, mientras que avanza se vuelve true ahora si presionamos la tecla G muevecarro se va a decrementar 1 y retrocede será true. Se puede observar el uso de GLFW\_\_RELEASE y esto básicamente es para que se detecte cuando se deja de presionar una tecla de esta manera avanza y retrocede cambian su valor a false puesto que significa que el carro no hace ninguna acción.

```
if (mainWindow.avanzar()) {
    Spottemp[s++] = spotLights[2];
    //printf("El valor de s es: %d\n", s);
    //s++;
    //printf("El valor de s es: %d\n", s);
}
//luz del faro trasero
if (mainWindow.retroceder()) {
    Spottemp[s++] = spotLights[3];
    //printf("El valor de s es: %d\n", s);
    //s++;
    //printf("El valor de s es: %d\n", s);
}
```

Ahora en el código de la practica lo que hacemos es usar estas condiciones si la función avanzar es true entonces que se mande esa luz al shader, caso contrario si retroceder es true entonces que se mande esa luz, ahora bien, GLFW\_\_RELEASE ayuda bastante para que esas luces no se queden prendidas, aunque ya no se este presionando la letra y con ayuda de las banderas cambiamos, así logramos que únicamente se mande ya sea una u la otra luz, depende de la acción.

Creamos y asignamos las luces al modelo del carro.

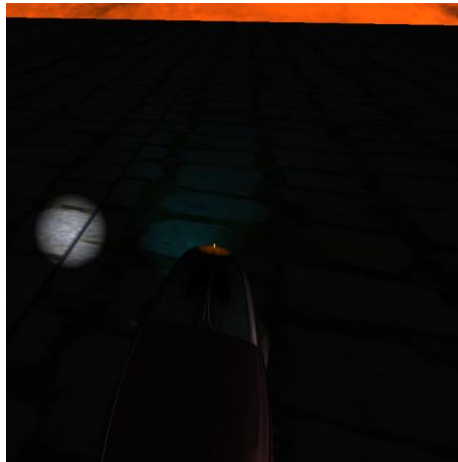
```
//FARO
spotLights[2] = SpotLight(
    0.00, 0.80, 0.80, // color
    1.0f, 2.0f, // intensidad
    4.7f, 1.35f, 6.4f, // posición inicial (se actualiza cada frame)
    1.0f, 0.0f, 0.0f, // dirección inicial (hacia adelante del carro)
    0.1f, 0.2f, 0.02f, // atenuación
    22.0f // tamaño de iluminacion
);

//luz trasera
spotLights[3] = SpotLight(
    1.00, 0.05, 0.05, // color
    1.0f, 2.0f, // intensidad
    0.0f, 0.0f, 0.0f, // posición inicial (se actualiza cada frame)
    -1.0f, 0.0f, 0.0f, // dirección inicial (hacia adelante del carro)
    0.1f, 0.2f, 0.05f, // atenuación
    22.0f // tamaño de iluminacion
);
```

```
//Instancia del coche
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getmuevecarro(), 1.8f, 8.0f));
modelaux = model;
spotLights[2].SetPos(glm::vec3(modelaux[3].x + 4.8f, modelaux[3].y - 0.5f, modelaux[3].z - 1.65f));
spotLights[3].SetPos(glm::vec3(modelaux[3].x - 4.8f, modelaux[3].y - 0.83f, modelaux[3].z - 1.77f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_M.RenderModel();
```

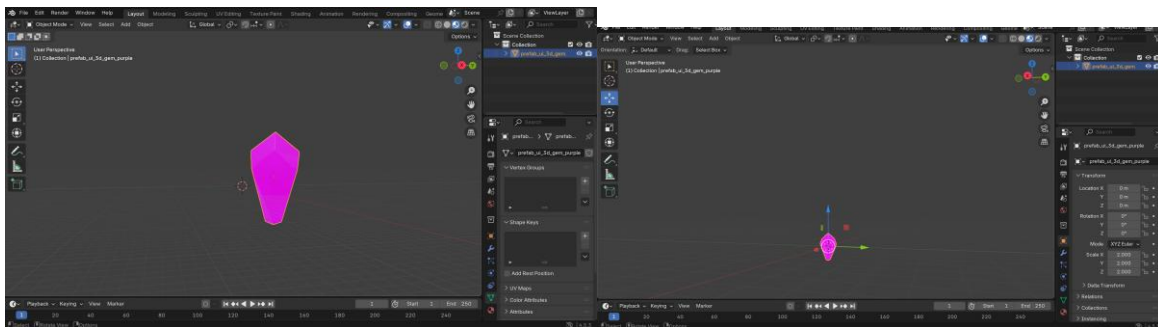
Ahora tenemos dos luces tanto la delantera como la trasera, de igual forma están por jerarquía gracias al uso de modelaux y con el SetPos la colocamos en la posición correcta y únicamente prendera la que se necesite gracias que solo envía cierta luz al shader.

## Ejecución



**3.- Agregar otra luz de tipo puntual ligada a un modelo elegido por ustedes ( no lámpara) y que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara como la luz de este modelo (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual).**

Como objeto utilice una gema que es del mundo de Crash Bandicoot que se obtienen en los mundos mientras juegas que es el objetivo principal, únicamente lo importe en Blender.



Como ya tenían texturas ya únicamente me encargue de revisar que estuviera bien y que se le aplicaran de manera correcta, y lo exporte como obj para hacer uso de el en el código al igual que la antorcha (que fue de la práctica pasada).

```
//iluminacion  
Model Antorcha_M;  
Model Gema_M;
```

```
Antorcha_M = Model();  
Antorcha_M.LoadModel("Models/antorcha.obj");  
  
Gema_M = Model();  
Gema_M.LoadModel("Models/gema_crash.obj");
```

Creamos dos funciones para el manejo de las luces luz1() y luz2() para distinguirlas pues únicamente cambiamos el valor de la variable que va a retornar,



entonces la primera se va a encargar de la luz antorcha y la segunda de la luz de la gema que es el objeto que se agregó.

```
bool luz1() { return luzantorcha; }
bool luz2() { return luzgema; }
```

```
//para cambio en luces puntuales
if (key == GLFW_KEY_O) {
    theWindow->luzantorcha = !theWindow->luzantorcha;
}
if (key == GLFW_KEY_P) {
    theWindow->luzgema = !theWindow->luzgema;
}
```

Ahora esto son valores únicamente booleanos y lo que logramos es que si presiono la tela O se quede la luz prendida de la antorcha si presiono P se quede la luz prendida de la gema, si vuelvo a presionar se apaga la luz, entonces así lo manejamos para que el control fuera de manera independiente.

```
spotLights[0] = SpotLight(1.0f, 1.0f, 1.0f,
    0.0f, 2.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    5.0f);
//spotLightCount++;
//se crean mas luces puntuales y spotlight

//ANTORCHA
pointLights[0] = PointLight(
    1.0, 1.0, 1.0f, // color: blanco
    0.2f, 0.9f, // intensidades (ambient, diffuse)
    0.0f, 0.0f, 0.0f, // posicion:
    0.3f, 0.1f, 0.1f // atenuación
);
//pointLightCount++;
```

Creamos las luces puntuales con sus valores iniciales.

```
//luz puntual de la antorcha
if (mainWindow.luz1()) {
    pointLightstemp[p++] = pointLights[0];
} //luz puntual de la gema

if (mainWindow.luz2()) {
    pointLightstemp[p++] = pointLights[1];
}
//mandamos al shader unicamente las luces que se van a ocupar
shaderList[0].SetPointLights(pointLightstemp, p);
```

En el código de la practica tenemos lo siguiente, aquí vemos que si los valores son true o verdaderos estos se van a enviar al shader así nuestras luces puntuales pueden estar de forma independiente o juntas a la vez ahora cabe resaltar que para que las luces puntuales no estuvieran en el shader el valor inicial es false.

```
//antorcha
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(10.0f, 1.5f, -2.0));
modelaux = model;
pointLights[0].SetPos(glm::vec3(modelaux[3].x - 0.1, modelaux[3].y + 1.4f, modelaux[3].z));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Antorcha_M.RenderModel();

//gema
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(8.0f, 1.5f, -2.0));
modelaux = model;
pointLights[1].SetPos(glm::vec3(modelaux[3].x - 0.1, modelaux[3].y + 1.4f, modelaux[3].z));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Gema_M.RenderModel();
```

Agregamos las luces para cada modelo y con el SetPos acomodamos las luces y con modelaux lo que hacemos es agregarlo por jerarquía a cada modelo.

```
luzantorcha = false;
luzgema = false;
```

Si las tuviéramos como true entonces inmediatamente al ejecutar el código entrarían a la función.

En general la inicialización de las variables es false para que no entren directamente al ejecutar si no hasta que se presionen las teclas.

## Ejecución





**2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.**

No hubo ningún problema al realizar los ejercicios.

### **3.- Conclusión:**

Con esta práctica se aplicó aún más el uso de las luces y sobre todo la optimización de las mismas enviando únicamente las que necesitábamos al shader, el ejercicio más complicado sin duda fue hacer que la luz siguiera el cofre entonces eso fue a mi punto de vista lo más difícil, de ahí en fuera solo signifíco agregar la lógica necesaria para que solo ciertas luces fueran utilizadas sin la necesidad de mandar todas, esta fue una práctica muy buena y que me ayudó mucho a entender como optimizar el uso de las luces.